CPU and device cooperation

Activities 1 and 2 | 11 minutes | Predict, observe, explain

Name: _____ Student ID: _____
Date: _____ Partner: _____

Use this worksheet during the Week 2 lesson. Allow about 55-65 minutes for the core activities and discussion. Optional experiments add about 12 minutes. These exercises are formative practice, with no additional graded submission.

Use an ordinary account in a prepared Linux environment with Bash. Run commands line by line. Output varies across machines. Record missing or restricted tools and use the stated fallback. Paper tasks require no Linux installation.

Activity 1 Read the timeline

Assume one CPU. Process A requests a device operation and waits. Process B is runnable. The intervals below are hypothetical time units.

Interval	CPU activity	Device activity
0-1	Submit request for A	Accept request
1-8	Run B	Transfer data
8-9	Handle completion	Complete request
After 9	Select runnable work	Available

1. When does A wait? What useful work can the CPU perform then?

2. Must A run immediately after completion? Explain.

3. How would CPU activity differ with a tight busy-wait loop?

Activity 2 Observe interrupt counters

```
cat /proc/interrupts
# Wait briefly, then run the command again.
cat /proc/interrupts
```

Compare the same row and CPU column. An empty or hidden interface does not prove that interrupts are absent. If unavailable, use Activity 1 to explain completion notification.

Row and CPU column: _____ First count: _____ Second: _____ Change: _____

What does this counter observation establish, and what does it leave unknown?

Cache behavior and access time

Activities 3 and 4 | 14 minutes | Paper models

Activity 3 Trace a small cache

A cache hit finds the requested block in the cache. A miss requires a lower-level access. Assume a fully associative cache that starts empty and holds two blocks. Each letter represents one block. Use least recently used (LRU) replacement. Every access updates recency.

For each reference, write H or M. List cached blocks from **least to most recently used**. When full, evict the least recent block.

Step	Reference	Hit or miss	Cache after access	Evicted block
1	A			
2	B			
3	A			
4	C			
5	A			
6	B			

Total hits: _____ Total misses: _____ Hit rate: _____

Repeat with three cache lines. How does the result change, and why?

Why does this model not guarantee the behavior of a real processor?

Activity 4 Calculate average memory access time

Use **AMAT = hit time + miss rate x additional miss penalty**. A hit takes 1 ns. A miss adds 100 ns after the cache check. Use decimal miss rates in the calculation. These are teaching values, not measurements.

Miss rate	Calculation	AMAT with units
5%		
2%		
10%		

If AMAT falls by half, must the entire program run twice as fast? Explain.

A program visits nearby array elements in sequence. Which kind of locality does this illustrate?

Memory and CPU topology

Activities 5 and 7 | 15 minutes | Linux observations

Activity 5 Distinguish the resources

```
lscpu
free -h
df -h .
```

lscpu reports visible CPU topology and may report cache information. **free** reports memory accounting. **df** reports filesystem space. **-h** requests readable units; the dot selects the filesystem containing the current directory.

Observation	Your result including units
RAM total, free and available	
Filesystem size, used and available	
One cache value and its exact label	

Why can available RAM differ from free RAM?

Does 100 GB of available filesystem space imply 100 GB of RAM? Explain.

Activity 7 Inspect packages cores and logical CPUs

```
lscpu -e=CPU,CORE,SOCKET,ONLINE
```

A package contains physical cores. SMT can expose several logical CPUs per core. Logical CPUs share core resources. If the extended command is unsupported, use the basic lscpu summary. Record unavailable fields as “not reported”.

Reported field	Value
CPU(s)	
Thread(s) per core	
Core(s) per socket	
Socket(s)	

Hypothetical uniform topology: 1 socket x 4 cores x 2 threads per core. Calculate logical CPUs.

Can VM output prove the physical host topology? What can container limits change?

The complete I O request

Activity 6 | 7 minutes | Pair explanation

A system call requests a kernel service. DMA moves data between a device and memory. An interrupt can notify a CPU of completion. A driver prepares and controls the device operation. A scheduler selects runnable work.

Case A Data already exists in the page cache

An application requests bytes from an ordinary buffered file. The requested file data is already available in RAM.

Describe how the kernel can satisfy the read. Is a new storage transfer required?

Case B Storage I O is required

Assume the requested data is absent from the page cache and a DMA-capable controller retrieves it. Complete the sequence using the appropriate role or event.

Step	Role or mechanism
Application requests a kernel service	
Kernel software prepares device operation	
Hardware transfers a block into RAM	
CPU receives completion notification	
Waiting caller becomes runnable	
Runnable caller receives CPU time	

While the caller waits, what can another runnable thread do?

Can a buffered read involving DMA still require a CPU copy into the user buffer?

Buffer lifetime

The following is conceptual pseudocode with invented function names. Do not enter it into Bash or compile it as an application.

```
address = map_buffer_for_device(buffer);
start_device_transfer(address, length);
wait_for_completion();
unmap_for_cpu_access(address);
process_data(buffer);
```

Why must the buffer remain valid until the transfer finishes?

Shared updates and waiting

Activity 8 | 6 minutes | Optional experiment A | 4 minutes

Activity 8 Explain a lost update

A shared counter starts at 5. Thread A and Thread B each intend to add one. Model each increment as read, calculate and store. Complete this interleaving on paper.

Step	Thread A	Thread B	Shared counter
1	Read counter		5
2		Read counter	
3	Calculate local value + 1		
4		Calculate local value + 1	
5	Store local value		
6		Store local value	

Expected result: _____ Observed result in this trace: _____ Why do they differ?

Rewrite the order so A completes its entire increment before B starts.

What shared operation needs protection? Name one suitable synchronization mechanism.

This is an illustrative trace. Unsynchronized conflicting accesses to an ordinary shared C object cause a data race and undefined behavior. This worksheet does not ask you to execute unsafe shared-counter code.

Optional experiment A Compare elapsed time and CPU time

```
time sleep 1
```

Bash reports real elapsed time, user CPU time and system CPU time. sleep performs a timed wait. This experiment does not show storage I/O or DMA.

Record real: _____ user: _____ sys: _____

Why can elapsed time be roughly one second while CPU time is small?

Can concurrency occur on one logical CPU? How does parallelism differ?

File reads and final explanation

Optional experiment B | 8 minutes | Exit ticket | 5 minutes

Create and read a small file

Keep one Bash session open. Run each line separately. If mktemp or cd fails, stop and resolve the error before writing the file. The workspace path is stored in work.

```
work=$(mktemp -d "$HOME/ccip504-week2.XXXXXX")
cd "$work"
pwd
printf "CPU and device cooperation\n" > sample.txt
cat sample.txt
cat sample.txt
```

mktemp -d creates a unique directory. **$(...)** captures its path. **printf** produces text and **>** writes it to the new file. **cat** displays the file. Record the workspace path if you need to return later.

Absolute workspace path:

Did both reads display the same content? Does that prove both accessed storage?

Optional system call trace

```
strace -e trace=openat,read,write,close cat sample.txt
```

Use this only if strace is installed and tracing is permitted. Otherwise mark it unavailable. Look for sample.txt among startup operations. A file descriptor is a process-local integer for an open I/O object. read and write return byte counts or errors.

Copy one relevant trace line and explain its return value, or record the access limitation.

Can this trace establish that a particular read used DMA? Explain.

A newly written tiny file may already be in the page cache. Repeated cat commands do not isolate storage speed or CPU cache latency. Keep the file as evidence; do not flush system caches.

Exit ticket

Explain the distinct roles of a system call, DMA and an interrupt in your own words.

State one result you observed and one thing it cannot establish.

Completion: Fill in the core activities, include units, mark unavailable tools and retain your observations as instructed. See the Week 2 lecture notes for model answers. Reading: Operating System Concepts, 10th edition, Chapter 1 and selected Chapter 12.